CSE4502 (Operating Systems Lab): 2A

Jibon Naher[1] and Samnun Azfar[2]

[1]Assistant Professor, CSE
[2]Junior Lecturer,CSE

February 26, 2026

Instruction

- Use the PC from the lab
- Total time is 40 minutes
- Submit your phone at the front
- Turn off the internet connection

1 Task: Preepmtive Priority Scheduling Algorithm

1.1 Given Code (Do NOT modify this part)

The following structure and main function are provided. Your implementation must integrate seamlessly with this boilerplate.

```cpp
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

struct Process {
    int pid;       // Process ID
    int at;        // Arrival Time
    int bt;        // Burst Time
    int priority;  // Priority (lower number = higher priority)
    int rt;        // Remaining Time
    int ct;        // Completion Time
    int tat;       // Turnaround Time
    int wt;        // Waiting Time
    bool completed; // Completion flag
};

// Function to perform Preemptive Priority Scheduling
void preepmtivePriorityScheduling(vector<Process>& p) {
    // TODO: Implement this function
}

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<Process> processes(n);

    for (int i = 0; i < n; i++) {
```

```
        processes[i].pid = i + 1;
        cout << "Enter Arrival Time, Burst Time, and Priority for Process "
             << processes[i].pid << ": ";
        cin >> processes[i].at >> processes[i].bt >> processes[i].priority;
    }

    // Call the function
    priorityScheduling(processes);

    return 0;
}
```

1.2 Implementation Hints

To successfully implement the `preepmtivePriorityScheduling` function.

- **Initialize:** Set `currentTime = 0` and keep track of completed processes.
- **At each time unit (increment time by 1):**
  1. **Selection:** Among all arrived (`at <= currentTime`) and incomplete (`remaining > 0`) processes, select the one with the **highest priority** (lowest priority value).
  2. **Tie-breaker:** If multiple processes have the same highest priority, pick the process with the **earliest arrival time**.
  3. **Execution:** Execute the selected process for **1 time unit**:
     - Decrement its `remaining` time.
     - Increment the `currentTime`.
  4. **Preemption Logic:** Because we re-evaluate the highest priority process at every single time unit, if a new process arrives with a higher priority (lower number), it will naturally be selected in the next iteration, effectively preempting the current one.
  5. **Completion:** If a process's `remaining` time reaches 0:
     - Record its Completion Time (`ct`).
     - Calculate $TAT = CT - AT$.
     - Calculate $WT = TAT - BT$.
  6. **Idle Handling:** If no process has arrived yet, increment the clock until the next arrival occurs.

1.3 Your Task

Complete the `preepmtivePriorityScheduling` function to fulfill the following requirements:

1. Simulate priority shceduling using the logic described above.
2. Print a table showing the ID, AT, BT, CT, TAT, and WT for every process.
3. Compute and display the **Average Turnaround Time** and **Average Waiting Time**.